

GUÍA DE ORGANIZACIÓN DEL PROYECTO

NexusPlay Hub

Plataforma de Reclutamiento para eSports

Miembro	Rol Principal
David	Frontend + Diseño
Daniel	Backend Laravel
Mario	Integraciones + Features

ÍNDICE

1. Descripción del Proyecto
2. Stack Tecnológico
3. División del Trabajo
 - 3.1. Detalle Técnico del Trabajo de David
 - 3.2. Detalle Técnico del Trabajo de Daniel
 - 3.3. Detalle Técnico del Trabajo de Mario
4. Metodología de Trabajo en Paralelo
5. Uso de GitHub y Control de Versiones
6. Estructura del Repositorio
7. Workflow de Git - Paso a Paso
8. Reglas de Oro del Equipo

1. DESCRIPCIÓN DEL PROYECTO

NexusPlay Hub es una plataforma profesional de reclutamiento para eSports que permite a jugadores y equipos conectar, gestionar tryouts y construir su legado en el mundo competitivo.

Funcionalidades principales:

- **Find Players:** Búsqueda de jugadores con filtros por juego, rango, región y estadísticas
- **Find Teams:** Equipos publicando ofertas de reclutamiento para roles específicos
- **Dashboard personalizado:** Actividad reciente, invitaciones, estadísticas del perfil
- **Sistema de mensajería:** Comunicación directa entre jugadores y equipos
- **Gestión de eventos:** Torneos, tryouts y competiciones
- **Blog de noticias:** Actualizaciones de parches, torneos y guías para jugadores

2. STACK TECNOLÓGICO

Componente	Tecnología	Responsable
Backend	Laravel (PHP)	Daniel
Frontend	React	Tú
Blog/CMS	WordPress (Headless)	Tú
API Externa	IGDB API (Twitch)	Mario
Base de Datos	MySQL	Daniel
Control de Versiones	Git + GitHub	Todos

WordPress como CMS Headless:

WordPress se utilizará exclusivamente como gestor de contenido para la sección de noticias/blog. Los editores crearán contenido desde el panel de administración de WordPress (wp-admin) y React consumirá este contenido mediante la API REST de WordPress (wp-json/wp/v2/posts). Esto permite cumplir con el requisito del profesor de usar WordPress mientras mantenemos una arquitectura moderna y desacoplada.

3. DIVISIÓN DEL TRABAJO

TÚ - Frontend + Diseño UX/UI

- Mockups completos en Figma de todas las páginas
- Desarrollo del frontend completo en React
- Componentes reutilizables (PlayerCard, TeamCard, Navbar, etc.)
- Páginas: Home, Dashboard, Find Players, Find Teams, Profile, Messages
- Integración de la API de WordPress para mostrar el blog
- Responsive design para móviles y tablets
- Testing del frontend

DANIEL - Backend Laravel

- Diseño de la base de datos (modelos, relaciones, migraciones)
- API REST completa en Laravel
- Sistema de autenticación (JWT o Sanctum)
- Endpoints principales: /api/players, /api/teams, /api/auth, /api/profile
- Validaciones y lógica de negocio
- Gestión de usuarios, equipos y perfiles
- Testing del backend

MARIO - Integraciones + Features Específicas

- Integración completa con IGDB API (obtener juegos, imágenes, información)
- Endpoints: /api/games, /api/games/search
- Sistema de estadísticas de jugadores (win rate, match history, rankings)
- Sistema de notificaciones (nuevos mensajes, invitaciones, matches)
- Sistema de mensajería entre usuarios (conversaciones, mensajes)
- Instalación y configuración de WordPress headless
- Testing de integraciones

NOTA: El trabajo de Mario está detallado extensamente en las páginas siguientes con ejemplos de código, estructura de bases de datos y pasos técnicos específicos.

3. DIVISIÓN DEL TRABAJO

DAVID - Frontend + Diseño UX/UI

- Mockups completos en Figma de todas las páginas
- Desarrollo del frontend completo en React
- Componentes reutilizables (PlayerCard, TeamCard, Navbar, etc.)
- Páginas: Home, Dashboard, Find Players, Find Teams, Profile, Messages
- Integración de la API de WordPress para mostrar el blog
- Responsive design para móviles y tablets
- Testing del frontend

DANIEL - Backend Laravel

- Diseño de la base de datos (modelos, relaciones, migraciones)
- API REST completa en Laravel
- Sistema de autenticación (JWT o Sanctum)
- Endpoints principales: /api/players, /api/teams, /api/auth, /api/profile
- Validaciones y lógica de negocio
- Gestión de usuarios, equipos y perfiles
- Testing del backend

MARIO - Integraciones + Features Específicas

- Integración completa con IGDB API (obtener juegos, imágenes, información)
- Endpoints: /api/games, /api/games/search
- Sistema de estadísticas de jugadores (win rate, match history, rankings)
- Sistema de notificaciones (nuevos mensajes, invitaciones, matches)
- Sistema de mensajería entre usuarios (conversaciones, mensajes)
- Instalación y configuración de WordPress headless
- Testing de integraciones

NOTA: El trabajo de cada miembro está detallado extensamente en las páginas siguientes con ejemplos de código, estructura de componentes/bases de datos y pasos técnicos específicos.

3.1. DETALLE TÉCNICO DEL TRABAJO DE DAVID

Esta sección detalla exhaustivamente las tareas de David con ejemplos de código React, estructura de componentes y mejores prácticas de diseño.

TAREA 1: Diseño de Mockups en Figma

Antes de escribir una sola línea de código, David debe crear los diseños completos en Figma. Esto permite validar el diseño con el equipo y tener una guía visual clara durante el desarrollo.

Páginas a diseñar:

- **Landing Page (Home):** Página pública con hero section, features, estadísticas, CTAs
- **Dashboard:** Panel personalizado con stats cards, actividad reciente, recomendaciones
- **Find Players:** Grid de PlayerCards con filtros laterales y barra de búsqueda
- **Find Teams:** Grid de TeamCards con información de reclutamiento
- **Profile:** Perfil del jugador con estadísticas, historial, badges
- **Messages:** Interface de chat tipo WhatsApp con lista de conversaciones
- **Blog/News:** Listado de artículos con preview y página de artículo individual

Elementos de diseño a definir:

- Paleta de colores (rojo #DC2626 como principal, grises para fondos)
- Tipografía (fuentes para títulos y cuerpo de texto)
- Iconografía (lucide-react o heroicons)
- Componentes reutilizables (botones, cards, inputs, modales)
- Estados (hover, active, disabled, loading)
- Responsive breakpoints (móvil 375px, tablet 768px, desktop 1440px)

TAREA 2: Estructura del Proyecto React

Organización de carpetas:

```
frontend/  
  public/  
    index.html  
  src/  
    components/ # Componentes reutilizables  
    Navbar.jsx  
    Footer.jsx  
    Button.jsx  
    SearchBar.jsx  
    PlayerCard.jsx  
    TeamCard.jsx  
    StatCard.jsx  
    ActivityItem.jsx  
    Modal.jsx
```

```
■ ■■■ pages/ # Páginas completas
■ ■ ■■■ Home.jsx
■ ■ ■■■ Dashboard.jsx
■ ■ ■■■ FindPlayers.jsx
■ ■ ■■■ FindTeams.jsx
■ ■ ■■■ Profile.jsx
■ ■ ■■■ Messages.jsx
■ ■ ■■■ Blog.jsx
■ ■
■ ■■■ data/ # Mock data (temporal)
■ ■ ■■■ mockPlayers.js
■ ■ ■■■ mockTeams.js
■ ■ ■■■ mockPosts.js
■ ■
■ ■■■ styles/
■ ■ ■■■ index.css
■ ■ ■■■ tailwind.css
■ ■
■ ■■■ App.jsx
■ ■■■ main.jsx
■
■■■ package.json
```


TAREA 3: Desarrollo de Componentes Reutilizables

Componente 1: PlayerCard

```
// src/components/PlayerCard.jsx
import React from 'react';

function PlayerCard({ player }) {
  return (
    <div className="bg-gray-800 rounded-lg p-6 hover:bg-gray-700 transition">
      <div className="flex flex-col items-center">
        <img
          src={player.avatar}
          alt={player.username}
          className="w-20 h-20 rounded-full mb-4"
        />
        <h3 className="text-xl font-bold text-white">{player.username}</h3>
        <p className="text-gray-400 text-sm">{player.realName}</p>

        <div className="flex gap-2 my-3">
          <span className="bg-red-600 px-3 py-1 rounded text-sm">
            {player.game}
          </span>
          <span className="bg-gray-700 px-3 py-1 rounded text-sm">
            {player.rank}
          </span>
        </div>

        <div className="w-full grid grid-cols-2 gap-4 my-4">
          <div>
            <p className="text-2xl font-bold text-white">{player.win_rate}%</p>
            <p className="text-gray-400 text-xs">Win Rate</p>
          </div>
          <div>
            <p className="text-2xl font-bold text-white">{player.hours}</p>
            <p className="text-gray-400 text-xs">Hours</p>
          </div>
        </div>

        <div className="flex gap-2 w-full">
          <button className="flex-1 bg-gray-700 hover:bg-gray-600 px-4 py-2 rounded">
            View Profile
          </button>
          <button className="flex-1 bg-red-600 hover:bg-red-700 px-4 py-2 rounded">
            Message
          </button>
        </div>
      </div>
    </div>
  );
}

export default PlayerCard;
```

Componente 2: TeamCard

```
// src/components/TeamCard.jsx
function TeamCard({ team }) {
  return (
    <div className="bg-gray-800 rounded-lg p-6">
      <div className="flex items-center gap-4 mb-4">
        <img src={team.logo} className="w-16 h-16 rounded" />
        <div>
          <h3 className="text-xl font-bold">{team.name}</h3>
          <span className="text-red-500">{team.game}</span>
          <span className="text-gray-400 ml-2">{team.level}</span>
        </div>
      </div>

      <p className="text-gray-400 mb-2">
        ■ {team.members}/5 Members
      </p>

      <p className="text-sm mb-4">
        Looking for: <span className="text-red-500">{team.looking_for}</span>
      </p>

      <div className="grid grid-cols-2 gap-4 mb-4">
```

```

<div>
<p className="text-xl font-bold">{team.win_rate}%</p>
<p className="text-xs text-gray-400">Win Rate</p>
</div>
<div>
<p className="text-xl font-bold">{team.tournaments}</p>
<p className="text-xs text-gray-400">Tournaments</p>
</div>
</div>

<div className="flex gap-2">
<button className="flex-1 bg-gray-700 px-4 py-2 rounded">
View Team
</button>
<button className="flex-1 bg-red-600 px-4 py-2 rounded">
Apply to Join
</button>
</div>
</div>
);
}

```

Componente 3: Navbar

```

// src/components/Navbar.jsx
import { Link } from 'react-router-dom';

function Navbar() {
return (
<nav className="bg-gray-900 border-b border-gray-800">
<div className="container mx-auto px-4 py-4 flex items-center justify-between">
<div className="flex items-center gap-8">
<Link to="/" className="flex items-center gap-2">
<span className="text-2xl font-bold">
<span className="text-red-600">N</span>exuPlay
</span>
</Link>

<div className="hidden md:flex gap-6">
<Link to="/find-players">Find Players</Link>
<Link to="/find-teams">Find Teams</Link>
<Link to="/jobs">Jobs</Link>
<Link to="/news">News</Link>
<Link to="/tryouts">Tryouts</Link>
</div>
</div>

<div className="flex items-center gap-4">
<button className="px-4 py-2 text-white hover:text-red-500">
Login
</button>
<button className="px-6 py-2 bg-red-600 rounded hover:bg-red-700">
Register
</button>
</div>
</div>
</nav>
);
}

```

TAREA 4: Desarrollo de Páginas Principales

Página: Find Players

```
// src/pages/FindPlayers.jsx
import { useState, useEffect } from 'react';
import PlayerCard from '../components/PlayerCard';
import { mockPlayers } from '../data/mockPlayers';

function FindPlayers() {
  const [players, setPlayers] = useState([]);
  const [filters, setFilters] = useState({
    game: '',
    rank: '',
    region: ''
  });

  useEffect(() => {
    // FASE 1: Usar mock data
    setPlayers(mockPlayers);

    // FASE 2: Cuando Daniel termine el backend
    // fetch('/api/players')
    // .then(res => res.json())
    // .then(data => setPlayers(data));
    //, []);

    const filteredPlayers = players.filter(player => {
      if (filters.game && player.game !== filters.game) return false;
      if (filters.rank && player.rank !== filters.rank) return false;
      return true;
    });

    return (
      <div className="container mx-auto px-4 py-8">
        <h1 className="text-4xl font-bold mb-8">Find Players</h1>

        {/* Filtros */}
        <div className="mb-8 flex gap-4">
          <select onChange={(e) => setFilters({...filters, game: e.target.value})}>
            <option value="">All Games</option>
            <option value="Valorant">Valorant</option>
            <option value="CS2">CS2</option>
          </select>
        </div>

        {/* Grid de jugadores */}
        <div className="grid grid-cols-1 md:grid-cols-2 lg:grid-cols-4 gap-6">
          {filteredPlayers.map(player => (
            <PlayerCard key={player.id} player={player} />
          ))}
        </div>
      </div>
    );
  }
}
```

TAREA 5: Integración con WordPress para el Blog

David debe consumir la API REST de WordPress para mostrar las noticias del blog en React.

Página de Blog:

```
// src/pages/Blog.jsx
import { useState, useEffect } from 'react';

function Blog() {
  const [posts, setPosts] = useState([]);
  const [loading, setLoading] = useState(true);

  useEffect(() => {
    fetch('http://tusitio.com/wp-json/wp/v2/posts?_embed')
      .then(res => res.json())
      .then(data => {
        setPosts(data);
        setLoading(false);
      })
      .catch(err => console.error(err));
  }, []);

  if (loading) return <div>Loading...</div>;

  return (
    <div className="container mx-auto px-4 py-8">
      <h1 className="text-4xl font-bold mb-8">Latest News</h1>

      <div className="grid grid-cols-1 md:grid-cols-3 gap-6">
        {posts.map(post => (
          <article key={post.id} className="bg-gray-800 rounded-lg overflow-hidden">
            { /* Imagen destacada */ }
            {post._embedded?.['wp:featuredmedia'] && (
              <img
                src={post._embedded['wp:featuredmedia'][0].source_url}
                alt={post.title.rendered}
                className="w-full h-48 object-cover"
              />
            )}

            <div className="p-6">
              <h2>
                className="text-xl font-bold mb-2"
                dangerouslySetInnerHTML={{ __html: post.title.rendered }}
              />

              <div>
                className="text-gray-400 mb-4"
                dangerouslySetInnerHTML={{ __html: post.excerpt.rendered }}
              />

              <a
                href={` /blog/${post.slug}`}
                className="text-red-500 hover:text-red-600"
              >
                Read more →
              </a>
            </div>
          </article>
        )
        )}
      </div>
    </div>
  );
}
```

TAREA 6: Responsive Design

David debe asegurarse de que todas las páginas funcionan perfectamente en móviles, tablets y desktop.

Breakpoints de Tailwind:

- **sm:** 640px (móviles grandes)
- **md:** 768px (tablets)
- **lg:** 1024px (laptops)
- **xl:** 1280px (desktop)
- **2xl:** 1536px (pantallas grandes)

Ejemplo de código responsive:

```
<div className="
grid
grid-cols-1 /* 1 columna en móvil */
md:grid-cols-2 /* 2 columnas en tablet */
lg:grid-cols-4 /* 4 columnas en desktop */
gap-6
">
{players.map(player => <PlayerCard ... />)}
</div>
```

TAREA 7: Testing del Frontend

- **Testing visual:** Verificar que todos los componentes se ven correctamente
- **Testing responsive:** Probar en diferentes tamaños de pantalla (Chrome DevTools)
- **Testing de navegación:** Verificar que todos los links funcionan
- **Testing de formularios:** Comprobar validaciones y envío de datos
- **Testing de rendimiento:** Verificar tiempos de carga con React DevTools
- **Testing de integración:** Verificar que los datos de la API se muestran correctamente

RESUMEN: Tiempo Estimado de Trabajo de David

Tarea	Horas Estimadas	Complejidad
Mockups en Figma	15-20h	Media
Setup React + Tailwind	3-5h	Baja
Componentes reutilizables	20-25h	Media-Alta
Páginas principales	25-30h	Alta
Integración WordPress	8-10h	Media
Responsive Design	10-15h	Media

Testing + Ajustes	10-12h	Media
TOTAL	91-117h	

3.2. DETALLE TÉCNICO DEL TRABAJO DE DANIEL

Esta sección detalla exhaustivamente las tareas de Daniel con ejemplos de código Laravel, estructura de bases de datos y arquitectura de la API.

TAREA 1: Diseño de la Base de Datos

Daniel debe diseñar el esquema completo de la base de datos antes de programar la API.

Tablas principales:

```
// Migration: create_users_table
Schema::create('users', function (Blueprint $table) {
    $table->id();
    $table->string('username')->unique();
    $table->string('email')->unique();
    $table->string('password');
    $table->string('real_name')->nullable();
    $table->enum('role', ['player', 'team', 'admin']);
    $table->string('avatar')->nullable();
    $table->string('region')->nullable(); // North America, Europe...
    $table->timestamps();
});

// Migration: create_player_profiles_table
Schema::create('player_profiles', function (Blueprint $table) {
    $table->id();
    $table->foreignId('user_id')->constrained()->onDelete('cascade');
    $table->text('bio')->nullable();
    $table->json('games')->nullable(); // ["Valorant", "CS2"]
    $table->json('roles')->nullable(); // ["Duelist", "Controller"]
    $table->boolean('looking_for_team')->default(false);
    $table->timestamps();
});

// Migration: create_teams_table
Schema::create('teams', function (Blueprint $table) {
    $table->id();
    $table->foreignId('owner_id')->constrained('users');
    $table->string('name')->unique();
    $table->string('tag')->unique(); // Ej: "FNC", "G2"
    $table->text('description')->nullable();
    $table->string('game'); // Valorant, CS2...
    $table->string('level'); // Pro, Semi-Pro, Amateur
    $table->integer('max_members')->default(5);
    $table->string('logo')->nullable();
    $table->timestamps();
});

// Migration: create_team_members_table
Schema::create('team_members', function (Blueprint $table) {
    $table->id();
    $table->foreignId('team_id')->constrained()->onDelete('cascade');
    $table->foreignId('user_id')->constrained()->onDelete('cascade');
    $table->string('role'); // Captain, Player, Coach
    $table->timestamp('joined_at');
    $table->timestamps();

    $table->unique(['team_id', 'user_id']);
});

// Migration: create_team_applications_table
Schema::create('team_applications', function (Blueprint $table) {
    $table->id();
    $table->foreignId('team_id')->constrained();
    $table->foreignId('user_id')->constrained();
    $table->text('message')->nullable();
    $table->enum('status', ['pending', 'accepted', 'rejected'])->default('pending');
    $table->timestamps();
});
```

TAREA 2: Sistema de Autenticación

Daniel debe implementar un sistema de autenticación robusto usando Laravel Sanctum o JWT.

Controlador de autenticación:

```
// app/Http/Controllers/AuthController.php
use Illuminate\Support\Facades\Hash;
use Illuminate\Validation\ValidationException;

class AuthController extends Controller {
    public function register(Request $request) {
        $validated = $request->validate([
            'username' => 'required|unique:users|max:50',
            'email' => 'required|email|unique:users',
            'password' => 'required|min:8|confirmed',
            'role' => 'required|in:player,team'
        ]);

        $user = User::create([
            'username' => $validated['username'],
            'email' => $validated['email'],
            'password' => Hash::make($validated['password']),
            'role' => $validated['role']
        ]);

        $token = $user->createToken('auth-token')->plainTextToken;

        return response()->json([
            'user' => $user,
            'token' => $token
        ], 201);
    }

    public function login(Request $request) {
        $request->validate([
            'email' => 'required|email',
            'password' => 'required'
        ]);

        $user = User::where('email', $request->email)->first();

        if (!$user || !Hash::check($request->password, $user->password)) {
            throw ValidationException::withMessages([
                'email' => ['The provided credentials are incorrect.']
            ]);
        }

        $token = $user->createToken('auth-token')->plainTextToken;

        return response()->json([
            'user' => $user,
            'token' => $token
        ]);
    }

    public function logout(Request $request) {
        $request->user()->currentAccessToken()->delete();
        return response()->json(['message' => 'Logged out successfully']);
    }
}
```

Rutas de autenticación:

```
// routes/api.php
Route::post('/auth/register', [AuthController::class, 'register']);
Route::post('/auth/login', [AuthController::class, 'login']);
Route::post('/auth/logout', [AuthController::class, 'logout']->middleware('auth:sanctum'));
```


TAREA 3: API de Jugadores

```
// app/Http/Controllers/PlayerController.php
class PlayerController extends Controller {
    public function index(Request $request) {
        $query = User::where('role', 'player')
        ->with('playerProfile');

        // Filtros opcionales
        if ($request->has('game')) {
            $query->whereHas('playerProfile', function($q) use ($request) {
                $q->whereJsonContains('games', $request->game);
            });
        }

        if ($request->has('region')) {
            $query->where('region', $request->region);
        }

        $players = $query->paginate(20);
        return response()->json($players);
    }

    public function show($id) {
        $player = User::with(['playerProfile', 'stats', 'matchHistory'])
        ->findOrFail($id);
        return response()->json($player);
    }

    public function update(Request $request, $id) {
        $user = User::findOrFail($id);

        // Verificar que el usuario autenticado es el propietario
        if (auth()->id() !== $user->id) {
            return response()->json(['error' => 'Unauthorized'], 403);
        }

        $validated = $request->validate([
            'real_name' => 'nullable|string|max:100',
            'bio' => 'nullable|string|max:500',
            'games' => 'nullable|array',
            'region' => 'nullable|string'
        ]);

        $user->update($validated);
        return response()->json($user);
    }
}
```

Rutas de jugadores:

```
Route::middleware('auth:sanctum')->group(function() {
    Route::get('/players', [PlayerController::class, 'index']);
    Route::get('/players/{id}', [PlayerController::class, 'show']);
    Route::put('/players/{id}', [PlayerController::class, 'update']);
});
```

TAREA 4: API de Equipos

```
// app/Http/Controllers/TeamController.php
class TeamController extends Controller {
    public function index(Request $request) {
        $query = Team::with(['owner', 'members']);

        if ($request->has('game')) {
            $query->where('game', $request->game);
        }

        $teams = $query->paginate(20);
        return response()->json($teams);
    }

    public function store(Request $request) {
        $validated = $request->validate([
            'name' => 'required|unique:teams|max:100',
            'tag' => 'required|unique:teams|max:10',
            'game' => 'required|string',
            'level' => 'required|in:Pro,Semi-Pro,Amateur',
            'description' => 'nullable|string'
        ]);

        $team = Team::create([
            ...$validated,
            'owner_id' => auth()->id()
        ]);

        // Añadir al creador como capitán
        TeamMember::create([
            'team_id' => $team->id,
            'user_id' => auth()->id(),
            'role' => 'Captain',
            'joined_at' => now()
        ]);

        return response()->json($team, 201);
    }

    public function applyToJoin(Request $request, $teamId) {
        $validated = $request->validate([
            'message' => 'nullable|string|max:500'
        ]);

        $application = TeamApplication::create([
            'team_id' => $teamId,
            'user_id' => auth()->id(),
            'message' => $validated['message'] ?? null
        ]);

        // Crear notificación para el owner del equipo
        // (Mario se encargará de esto)

        return response()->json($application, 201);
    }
}
```

TAREA 5: Validaciones y Seguridad

Daniel debe implementar validaciones robustas y medidas de seguridad:

- **Request validation:** Validar todos los inputs con reglas de Laravel
- **Middleware de autenticación:** Proteger rutas que requieren login
- **Rate limiting:** Limitar número de peticiones por IP
- **CORS:** Configurar correctamente para permitir requests desde React
- **SQL Injection:** Usar Eloquent ORM (previene automáticamente)
- **XSS:** Sanitizar datos antes de guardar en BD
- **Autorización:** Verificar que usuarios solo editan sus propios recursos

Ejemplo de middleware de autorización:

```
// app/Http/Middleware/EnsureOwnership.php
public function handle($request, Closure $next) {
    $resourceId = $request->route('id');
    $resource = User::findOrFail($resourceId);

    if (auth()->id() !== $resource->id) {
        return response()->json(['error' => 'Unauthorized'], 403);
    }

    return $next($request);
}
```

TAREA 6: Testing del Backend

- **Unit tests:** Probar modelos y relaciones
- **Feature tests:** Probar endpoints completos (register, login, CRUD)
- **Testing con Postman:** Crear colección de requests para pruebas manuales
- **Validar respuestas JSON:** Verificar estructura correcta de datos
- **Testing de errores:** Comprobar que errores devuelven códigos HTTP correctos
- **Testing de autenticación:** Verificar que rutas protegidas no son accesibles sin token

Endpoints que Daniel debe crear:

Método	Endpoint	Descripción
POST	/api/auth/register	Registrar nuevo usuario
POST	/api/auth/login	Iniciar sesión
POST	/api/auth/logout	Cerrar sesión
GET	/api/players	Listar jugadores (con filtros)

GET	/api/players/{id}	Ver perfil de jugador
PUT	/api/players/{id}	Editar perfil
GET	/api/teams	Listar equipos
POST	/api/teams	Crear equipo
GET	/api/teams/{id}	Ver equipo
POST	/api/teams/{id}/apply	Aplicar a equipo
GET	/api/profile	Ver perfil propio
PUT	/api/profile	Editar perfil propio

RESUMEN: Tiempo Estimado de Trabajo de Daniel

Tarea	Horas Estimadas	Complejidad
Diseño de BD (migraciones)	10-15h	Media-Alta
Setup Laravel + Sanctum	4-6h	Baja-Media
Sistema de autenticación	12-15h	Alta
API de jugadores	15-20h	Alta
API de equipos	15-20h	Alta
Validaciones y seguridad	8-10h	Media
Testing + Documentación	10-12h	Media
TOTAL	74-98h	

3.3. DETALLE TÉCNICO DEL TRABAJO DE MARIO

Esta sección detalla exhaustivamente las tareas de Mario con ejemplos de código, estructura de bases de datos y pasos técnicos específicos.

TAREA 1: Integración con IGDB API

¿Qué es IGDB?

IGDB (Internet Game Database) es una base de datos completa de videojuegos propiedad de Twitch. Proporciona información detallada sobre miles de juegos: títulos, géneros, plataformas, imágenes, fechas de lanzamiento, etc.

Pasos para la integración:

1. Obtener credenciales de Twitch:

- Registrarse en dev.twitch.tv
- Crear una aplicación
- Obtener Client ID y Client Secret
- Guardar en archivo .env de Laravel

2. Crear servicio en Laravel:

```
// app/Services/IgdbService.php
class IgdbService {
    private $clientId;
    private $accessToken;

    public function __construct() {
        $this->clientId = env('IGDB_CLIENT_ID');
        // Obtener access token de Twitch
    }

    public function searchGames($query) {
        $response = Http::withHeaders([
            'Client-ID' => $this->clientId,
            'Authorization' => 'Bearer ' . $this->accessToken
        ])->post('https://api.igdb.com/v4/games', [
            'search' => $query,
            'fields' => 'name,cover.url,genres.name'
        ]);
        return $response->json();
    }
}
```

3. Crear controlador y rutas:

```
// app/Http/Controllers/GameController.php
class GameController extends Controller {
    public function index(IgdbService $igdb) {
        $games = $igdb->getPopularGames();
        return response()->json($games);
    }

    public function search(Request $request, IgdbService $igdb) {
        $results = $igdb->searchGames($request->q);
        return response()->json($results);
    }
}

// routes/api.php
Route::get('/games', [GameController::class, 'index']);
Route::get('/games/search', [GameController::class, 'search']);
```

4. Cachear las respuestas (importante para no exceder límites):

```
public function searchGames($query) {
    return Cache::remember('games_' . $query, 3600, function() use ($query) {
        // Llamada a IGDB API
        return $this->makeApiCall($query);
    });
}
```

Endpoints que Mario debe crear:

- GET /api/games → Lista de juegos populares
- GET /api/games/search?q=valorant → Buscar juegos
- GET /api/games/{id} → Detalle de un juego específico
- GET /api/games/{id}/cover → Obtener imagen de portada

TAREA 2: Sistema de Estadísticas de Jugadores

Estructura de base de datos necesaria:

```
// Migration: create_player_stats_table
Schema::create('player_stats', function (Blueprint $table) {
    $table->id();
    $table->foreignId('user_id')->constrained();
    $table->string('game'); // Valorant, CS2, LoL...
    $table->string('rank'); // Immortal, Global Elite...
    $table->integer('win_rate'); // Porcentaje
    $table->integer('hours_played');
    $table->integer('matches_played');
    $table->timestamps();
});

// Migration: create_match_history_table
Schema::create('match_history', function (Blueprint $table) {
    $table->id();
    $table->foreignId('user_id')->constrained();
    $table->string('game');
    $table->boolean('won'); // true = victoria, false = derrota
    $table->string('map')->nullable();
    $table->integer('kills')->nullable();
    $table->integer('deaths')->nullable();
    $table->timestamp('played_at');
    $table->timestamps();
});
```

Controlador de estadísticas:

```
// app/Http/Controllers/StatsController.php
class StatsController extends Controller {
    public function show($userId) {
        $stats = PlayerStat::where('user_id', $userId)->get();
        $history = MatchHistory::where('user_id', $userId)
            ->orderBy('played_at', 'desc')
            ->limit(10)
            ->get();
        return response()->json([
            'stats' => $stats,
            'recent_matches' => $history
        ]);
    }

    public function update(Request $request) {
        // Actualizar stats después de una partida
        PlayerStat::updateOrCreate([
            'user_id' => auth()->id(),
            'game' => $request->game
        ], [
            'win_rate' => $this->calculateWinRate($request),
        ]);
    }
}
```

```
'hours_played' => $request->hours,  
'matches_played' => $request->matches  
]);  
}  
}
```

Endpoints de estadísticas:

- GET /api/players/{id}/stats → Obtener estadísticas de un jugador
- POST /api/stats → Actualizar estadísticas (después de partida)
- GET /api/players/{id}/matches → Historial de partidas
- GET /api/leaderboard/{game} → Ranking de jugadores por juego

TAREA 3: Sistema de Notificaciones

Las notificaciones informan a los usuarios sobre eventos importantes: nuevos mensajes, invitaciones a equipos, matches encontrados, etc.

Base de datos:

```
Schema::create('notifications', function (Blueprint $table) {
    $table->id();
    $table->foreignId('user_id')->constrained();
    $table->string('type'); // 'message', 'team_invite', 'match'
    $table->string('title');
    $table->text('content');
    $table->boolean('read')->default(false);
    $table->json('data')->nullable(); // Datos extra
    $table->timestamps();
});
```

Modelo y lógica:

```
// app/Models/Notification.php
class Notification extends Model {
    protected $fillable = ['user_id', 'type', 'title', 'content', 'read', 'data'];
    protected $casts = ['data' => 'array', 'read' => 'boolean'];

    public function user() {
        return $this->belongsTo(User::class);
    }
}

// app/Http/Controllers/NotificationController.php
class NotificationController extends Controller {
    public function index() {
        $notifications = auth()->user()->notifications()
            ->orderBy('created_at', 'desc')
            ->get();
        return response()->json($notifications);
    }

    public function markAsRead($id) {
        $notification = Notification::findOrFail($id);
        $notification->update(['read' => true]);
        return response()->json(['success' => true]);
    }
}
```

Crear notificaciones automáticamente:

```
// Ejemplo: Cuando alguien envía un mensaje
public function sendMessage(Request $request) {
    $message = Message::create([...]);

    // Crear notificación para el receptor
    Notification::create([
        'user_id' => $request->to_user_id,
        'type' => 'message',
        'title' => 'New message',
        'content' => auth()->user()->username . ' sent you a message',
        'data' => ['message_id' => $message->id]
    ]);
}
```

Endpoints de notificaciones:

- GET /api/notifications → Listar notificaciones del usuario
- PUT /api/notifications/{id}/read → Marcar como leída
- DELETE /api/notifications/{id} → Eliminar notificación
- GET /api/notifications/unread-count → Contador de no leídas

TAREA 4: Sistema de Mensajería

Sistema de chat 1-a-1 entre jugadores y equipos. Permite comunicación directa sin salir de la plataforma.

Estructura de base de datos:

```
// Tabla de conversaciones
Schema::create('conversations', function (Blueprint $table) {
    $table->id();
    $table->foreignId('user1_id')->constrained('users');
    $table->foreignId('user2_id')->constrained('users');
    $table->timestamp('last_message_at')->nullable();
    $table->timestamps();
});

// Tabla de mensajes
Schema::create('messages', function (Blueprint $table) {
    $table->id();
    $table->foreignId('conversation_id')->constrained();
    $table->foreignId('sender_id')->constrained('users');
    $table->text('content');
    $table->boolean('read')->default(false);
    $table->timestamps();
});
```

Controlador de mensajería:

```
class MessageController extends Controller {
    public function getConversations() {
        $userId = auth()->id();
        $conversations = Conversation::where('user1_id', $userId)
            ->orWhere('user2_id', $userId)
            ->with(['messages' => function($q) {
                $q->latest()->limit(1);
            }])
            ->orderBy('last_message_at', 'desc')
            ->get();
        return response()->json($conversations);
    }

    public function getMessages($conversationId) {
        $messages = Message::where('conversation_id', $conversationId)
            ->orderBy('created_at', 'asc')
            ->get();
        return response()->json($messages);
    }

    public function send(Request $request) {
        // Buscar o crear conversación
        $conversation = Conversation::firstOrCreate([...]);

        // Crear mensaje
        $message = Message::create([
            'conversation_id' => $conversation->id,
            'sender_id' => auth()->id(),
            'content' => $request->content
        ]);

        // Actualizar timestamp de conversación
        $conversation->update(['last_message_at' => now()]);

        return response()->json($message);
    }
}
```

Endpoints de mensajería:

- GET /api/conversations → Listar conversaciones del usuario
- GET /api/conversations/{id}/messages → Mensajes de una conversación

- POST /api/messages → Enviar un nuevo mensaje
- PUT /api/messages/{id}/read → Marcar mensaje como leído

TAREA 5: Instalación y Configuración de WordPress Headless

WordPress se usará como CMS para gestionar el blog de noticias. React consumirá el contenido mediante la API REST de WordPress.

Pasos de instalación:

1. Descargar WordPress desde wordpress.org
2. Crear base de datos MySQL para WordPress
3. Configurar wp-config.php con credenciales de BD
4. Completar instalación en navegador
5. Instalar tema básico (cualquiera, no se verá en frontend)
6. Configurar permalinks en 'Post name' (wp-admin → Settings → Permalinks)

Habilitar CORS en WordPress (importante):

```
// Añadir en wp-config.php o en functions.php del tema:
add_action('rest_api_init', function() {
    remove_filter('rest_pre_serve_request', 'rest_send_cors_headers');
    add_filter('rest_pre_serve_request', function($value) {
        header('Access-Control-Allow-Origin: *');
        header('Access-Control-Allow-Methods: GET, POST');
        return $value;
    });
}, 15);
```

Crear contenido de prueba:

- Crear 5-10 posts de ejemplo desde wp-admin → Posts → Add New
- Usar categorías: 'Torneos', 'Parches', 'Guías', 'Noticias'
- Añadir imágenes destacadas a cada post
- Publicar los posts

Endpoints de WordPress disponibles:

- GET /wp-json/wp/v2/posts → Lista de posts
- GET /wp-json/wp/v2/posts/{id} → Post individual
- GET /wp-json/wp/v2/categories → Lista de categorías
- GET /wp-json/wp/v2/posts?categories={id} → Posts de una categoría

Ejemplo de consumo desde React:

```
// En React
const [posts, setPosts] = useState([]);

useEffect(() => {
    fetch('http://tusitio.com/wp-json/wp/v2/posts')
        .then(res => res.json())
        .then(data => setPosts(data));
}, []);

return (
    <div>
```

```
{posts.map(post => (  
<article key={post.id}>  
<h2>{post.title.rendered}</h2>  
<div dangerouslySetInnerHTML={{__html: post.excerpt.rendered}} />  
</article>  
))}  
</div>  
);
```

TAREA 6: Testing de Integraciones

Mario debe asegurarse de que todas sus integraciones funcionan correctamente:

- **IGDB API:** Probar búsquedas, verificar que las imágenes se cargan, cachear correctamente
- **Estadísticas:** Crear datos de prueba, calcular win rates correctamente, mostrar rankings
- **Notificaciones:** Crear notificaciones de prueba, marcar como leídas, eliminar
- **Mensajería:** Enviar mensajes de prueba, verificar orden cronológico, marcar como leído
- **WordPress:** Verificar que los posts se obtienen correctamente desde React, probar categorías

RESUMEN: Tiempo Estimado de Trabajo de Mario

Tarea	Horas Estimadas	Complejidad
Integración IGDB API	15-20h	Media-Alta
Sistema de Estadísticas	10-15h	Media
Sistema de Notificaciones	8-12h	Media
Sistema de Mensajería	15-20h	Alta
WordPress Headless	5-8h	Baja
Testing + Documentación	8-10h	Media
TOTAL	61-85h	

NOTA IMPORTANTE: Este es trabajo real, medible y esencial para el proyecto. Mario no puede decir que 'solo ayuda' - tiene responsabilidades concretas y features que deben funcionar en la demo final. En la defensa del TFG podrá explicar perfectamente qué ha hecho y cómo funciona cada parte.

4. METODOLOGÍA DE TRABAJO EN PARALELO

PROBLEMA A RESOLVER: El frontend necesita datos del backend para funcionar, pero no podemos esperar a que Daniel termine todo el backend antes de empezar con React. Esto causaría retrasos enormes en el proyecto.

SOLUCIÓN: API Contract + Mock Data

Paso 1: Crear el Contrato de API (30 minutos, los 3 juntos)

El 'contrato' es un documento donde definís exactamente cómo van a ser los datos que el backend va a devolver. Esto permite que cada uno trabaje de forma independiente siguiendo el mismo esquema.

Ejemplo de contrato:

```
=== JUGADORES ===
Endpoint: GET /api/players
Respuesta:
[
  {
    "id": 1,
    "username": "Shadow",
    "game": "Valorant",
    "rank": "Immortal",
    "win_rate": 68,
    "hours": 2450
  }
]
```

Paso 2: Trabajar con Datos Falsos (Mock Data)

Mientras Daniel desarrolla el backend real, David crea archivos JavaScript con datos inventados que siguen EXACTAMENTE la misma estructura del contrato. Esto permite desarrollar todo el frontend sin esperar.

Ejemplo en React:

```
// src/data/mockPlayers.js
export const mockPlayers = [
  {
    id: 1,
    username: "Shadow",
    game: "Valorant",
    rank: "Immortal",
    win_rate: 68,
    hours: 2450
  },
  // Más jugadores inventados...
];
```

Paso 3: Conectar con la API Real (cuando Daniel termine)

Cuando Daniel tenga el backend funcionando, solo necesitas cambiar 2-3 líneas de código para pasar de los datos falsos a la API real:

```
// ANTES (datos falsos):
const [players, setPlayers] = useState(mockPlayers);

// DESPUÉS (API real):
useEffect(() => {
  fetch('http://localhost:8000/api/players')
    .then(res => res.json())
```

```
.then(data => setPlayers(data));  
}, []);
```

5. USO DE GITHUB Y CONTROL DE VERSIONES

Estructura de Ramas (Git Flow Simplificado)

```
main (o master) → Código 100% funcional para mostrar al profesor
↓
■■■ develop → Rama principal de trabajo del equipo
↓
■■■ feature/frontend-dashboard → TÚ
■■■ feature/frontend-players → TÚ
■■■ feature/backend-auth → DANIEL
■■■ feature/backend-players → DANIEL
■■■ feature/igdb-integration → MARIO
■■■ feature/messaging → MARIO
```

Explicación de cada rama:

- **main:** Solo código 100% funcional. Se actualiza al final de cada sprint o entrega.
- **develop:** Donde se integra el trabajo de todos. Es la rama 'de trabajo en equipo'.
- **feature/nombre:** Cada persona trabaja en su propia rama sin interferir con los demás.

Ventajas de este sistema:

- **Trabajo paralelo:** Cada uno avanza sin bloquear a los demás
- **Código organizado:** Fácil ver quién hizo qué y cuándo
- **Menos conflictos:** Al trabajar en ramas separadas, raramente se pisan el código
- **Rollback fácil:** Si algo falla, se puede volver atrás sin perder todo el trabajo
- **Code reviews:** Los Pull Requests permiten revisar código antes de integrarlo

6. ESTRUCTURA DEL REPOSITORIO

```
nexusplay/
├── backend/ ← DANIEL y MARIO trabajan aquí
│   ├── app/
│   ├── routes/
│   ├── database/
│   ├── config/
│   └── ...
├── frontend/ ← TÚ trabajas aquí
│   ├── src/
│   │   ├── components/ (PlayerCard, TeamCard, Navbar...)
│   │   ├── pages/ (Home, Dashboard, FindPlayers...)
│   │   ├── data/ (mockPlayers.js, mockTeams.js...)
│   │   ├── App.jsx
│   │   ├── public/
│   │   └── package.json
│   └── wordpress/ ← Instalación de WordPress
├── docs/ ← Documentación del proyecto
│   └── API_Contract.md (el contrato de la API)
├── .gitignore
└── README.md
```

Ventaja de esta estructura:

Al trabajar en carpetas diferentes (frontend/ vs backend/), los conflictos en Git son prácticamente inexistentes. Cada uno puede trabajar sin pisar el código de los demás.

Archivo .gitignore (IMPORTANTE)

Este archivo le dice a Git qué NO debe subir al repositorio. Es crucial para no subir archivos innecesarios o sensibles:

```
# Node modules
frontend/node_modules/
frontend/build/
frontend/.env

# Laravel
backend/vendor/
backend/.env
backend/storage/*.key

# IDEs
.vscode/
.idea/

# WordPress
wordpress/wp-config.php
wordpress/wp-content/uploads/
```


7. WORKFLOW DE GIT - PASO A PASO

Configuración Inicial (una sola vez)

1. Uno de vosotros (Daniel) crea el repositorio en GitHub
2. Los demás lo clonan:

```
git clone https://github.com/usuario/nexusplay.git
cd nexusplay
git checkout develop
```

Workflow Diario (cada vez que trabajáis)

PASO 1: Actualizar develop antes de empezar

```
git checkout develop
git pull origin develop
```

PASO 2: Crear una rama para tu nueva funcionalidad

```
git checkout -b feature/frontend-dashboard
# Ahora estás en tu rama personal
```

PASO 3: Trabajar normalmente

Haces cambios en tu código, creas archivos, editas componentes, etc.

PASO 4: Guardar cambios (commit)

```
git add .
git commit -m "Add dashboard layout with stats cards"
# Usa mensajes descriptivos, no "cambios" o "asdfasdf"
```

PASO 5: Subir tu rama a GitHub

```
git push origin feature/frontend-dashboard
```

PASO 6: Crear Pull Request cuando termines

1. Ve a GitHub → pestaña 'Pull Requests'
2. Click en 'New Pull Request'
3. Base: develop ← Compare: feature/frontend-dashboard
4. Añade un título descriptivo y una descripción de los cambios
5. Asigna a otro del equipo para que lo revise
6. Cuando lo aprueben → Merge to develop
7. Borra la rama feature/ después del merge

8. REGLAS DE ORO DEL EQUIPO

■ LO QUE SÍ DEBÉIS HACER:

- **SIEMPRE** trabajar en vuestra propia rama feature/
- **SIEMPRE** hacer 'git pull origin develop' antes de crear una rama nueva
- **Commits pequeños y frecuentes** con mensajes claros ('Add player search filters', no 'cambios')
- **Push** vuestra rama a GitHub frecuentemente, aunque no esté terminada
- **Pull Requests** cuando terminéis una funcionalidad completa
- **Code reviews:** Revisar el código de los demás antes de aprobar un PR
- **Comunicación:** Avisar en el grupo si vais a tocar algo sensible
- **Documentar:** Comentar el código complejo para que los demás lo entiendan

■ LO QUE NO DEBÉIS HACER NUNCA:

- **NUNCA** trabajar directamente en develop o main
- **NUNCA** hacer merge sin Pull Request
- **NUNCA** hacer 'git push -f' (force push) ← esto ROMPE el repositorio
- **NUNCA** commitear node_modules/, .env, o archivos de IDEs
- **NUNCA** hacer commits con mensajes vagos ('cambios', 'fix', 'asdf')
- **NUNCA** borrar ramas de otros sin preguntar
- **NUNCA** modificar el .gitignore sin avisar al equipo

■ REUNIÓN SEMANAL (15 minutos)

Cada lunes a las 10:00, reunión rápida por Discord/Teams para:

- ¿Qué hiciste la semana pasada?
- ¿Qué vas a hacer esta semana?
- ¿Tienes algún bloqueo o necesitas ayuda?
- Revisar Pull Requests pendientes
- Actualizar el tablero de tareas (Trello/Notion)

■ CRONOGRAMA SUGERIDO

Semanas	Actividades
1-2	Definir API Contract Mockups en Figma Setup inicial (repos, Laravel, React)
3-4	Frontend con mock data (70%) Backend endpoints básicos Integración IGDB inicial
5	Conexión frontend-backend Primeras pruebas integradas Ajustes y correcciones
6-7	WordPress headless Pulir diseño y UX Testing completo Documentación TFG
8	Preparar presentación Ensayar defensa Últimos ajustes

CONCLUSIÓN

Esta guía proporciona un marco de trabajo profesional y eficiente para completar el proyecto NexusPlay Hub. La clave del éxito está en:

- **Comunicación constante:** Usar Discord/WhatsApp diariamente para dudas rápidas
- **Trabajo paralelo:** Usar mock data para no bloquearse entre vosotros
- **Git disciplinado:** Seguir el workflow de ramas y Pull Requests rigurosamente
- **Responsabilidad individual:** Cada uno tiene tareas claras y medibles
- **Flexibilidad:** Si alguien se adelanta, puede ayudar en otras áreas
- **Documentación continua:** Ir escribiendo la memoria del TFG durante el desarrollo

Recordad que este proyecto no solo es para aprobar el TFG, sino para demostrar que sabéis trabajar en equipo usando metodologías profesionales que se aplican en empresas reales. ¡Mucho ánimo con el proyecto!